MATERIA

DESARROLLO DE VIDEOJUEGOS (INF761)

UNIDAD 4

Inteligencia Artificial para Videojuegos

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

4.1 Fundamentos de IA para juegos
Agentes, percepción, toma de decisiones y arquitecturas

4.2 FSM y Behavior Trees para NPCs
Máquinas de estados, árboles de comportamiento, comparativa

4.3 Navegación y pathfinding
NavMesh, NavMeshAgent, obstáculos dinámicos y A conceptual*

4.4 Sistemas de detección
Línea de visión, conos FOV, estados de alerta y patrullaje

4.5 IA avanzada
Utility AI, GOAP y steering behaviors para movimiento autónomo

4.6 Prototipo 3: Stealth/Acción con NPCs
Patrullaje, detección, persecución y combate inteligente

COMPETENCIA DE LA UNIDAD

Programar agentes inteligentes con capacidad de navegación, percepción y toma de decisiones utilizando FSM, Behavior Trees, NavMesh y técnicas avanzadas de IA para juegos.

4. Inteligencia Artificial para Videojuegos

4.1 Fundamentos de IA para juegos: agentes, percepción del entorno, toma de decisiones y arquitecturas comunes

La Inteligencia Artificial en videojuegos no busca crear agentes verdaderamente inteligentes, sino agentes que parezcan inteligentes al jugador. Un buen NPC no necesita resolver problemas de forma óptima; necesita comportarse de forma creíble, predecible en lo suficiente para que el jugador aprenda a interactuar con él, e impredecible en lo suficiente para que el juego sea interesante.

El agente y su entorno

En IA para juegos, un agente es cualquier entidad que percibe su entorno y toma decisiones autónomas en consecuencia. El ciclo fundamental de todo agente es:

- **Percepción:** El agente recopila información del mundo (posición del jugador, ruidos, objetos visibles, estado de salud propio).
- **Razonamiento:** El agente procesa la información y decide qué hacer (perseguir, huir, atacar, patrullar, llamar refuerzos).
- **Acción:** El agente ejecuta la decisión (mover hacia un punto, disparar, reproducir una animación, emitir un sonido).

DEFINICIÓN — AGENTE REACTIVO VS DELIBERATIVO

Un agente reactivo responde directamente a estímulos sin planificación (si ve al jugador → ataca). Es rápido y predecible. Un agente deliberativo mantiene un modelo interno del mundo y planifica secuencias de acciones para alcanzar objetivos (GOAP). Es más costoso computacionalmente pero produce comportamientos más complejos. La mayoría de NPCs de juegos comerciales son reactivos con capas de deliberación limitada.

Percepción del entorno

La percepción es la forma en que el agente recopila información del mundo del juego. Las técnicas más comunes son:

Técnica de percepción	Implementación en Unity	Uso típico
Visual (Line of Sight)	Raycast + OverlapCircle en cono de visión	Detectar jugador dentro del FOV
Auditiva (Hearing)	Radius check + variable de ruido del jugador	Escuchar pasos, disparos, explosiones
Táctil (Touch)	OnTriggerEnter con collider de detección	Detectar contacto físico directo
Memoria	Guardar última posición conocida del jugador	Investigar zona tras perder de vista
Comunicación	Evento/mensaje entre agentes	NPC alerta a sus compañeros

Arquitecturas de IA comunes en juegos

Arquitectura	Descripción	Ejemplos de uso
FSM (Finite State Machine)	Estados discretos con transiciones condicionales	Enemigos simples, puertas, trampas
Behavior Tree	Árbol jerárquico de nodos de control y acción	NPCs complejos, jefes, aliados
Utility AI	Puntuación de utilidad por acción, elige la mayor	Sims, estrategia, RPG con variedad
GOAP	Planificación de acciones hacia un objetivo	F.E.A.R., juegos de sigilo complejos
Steering Behaviors	Fuerzas vectoriales para movimiento autónomo fluido	Boids, manadas, multitudes
NavMesh + Pathfinding	Navegación sobre malla de navegación precalculada	Todos los juegos con NPCs que se mueven
Machine Learning (ML-Agents)	Entrenamiento por refuerzo para comportamientos emergentes	Investigación, juegos experimentales

4.2 Máquinas de estados finitos (FSM) y árboles de comportamiento (Behavior Trees) para NPCs

PROGRESIÓN DE CONCEPTOS — U2 → U3 → U4

En U2.2 se implementó una FSM manual en código (enum + switch) para controlar la IA de un enemigo simple. En U3.2 se vio el Animator Controller de Unity como una FSM visual especializada en animaciones. En esta sección se formalizan ambos conceptos y se introduce el Behavior Tree como evolución natural de la FSM para NPCs más complejos: donde la FSM escala mal (muchos estados = muchas transiciones cruzadas), el BT ofrece estructura jerárquica y reutilización de subárboles.

FSM — Máquina de estados finitos

Una FSM define un conjunto finito de estados y las condiciones que provocan transiciones entre ellos. En cada momento el agente está exactamente en un estado, y solo puede cambiar al estado siguiente si se cumple la condición de transición.

- **Ventajas:** Simple de implementar, fácil de depurar, predecible, bajo costo computacional.
- **Desventajas:** El número de transiciones crece cuadráticamente con los estados (n estados → hasta n² transiciones). Con más de 7-8 estados se vuelve difícil de mantener.

Implementación con interfaz IState para mayor flexibilidad y separación de responsabilidades:

```
// Interfaz base para todos los estados
public interface IState
{
    void Enter();
    void Tick();       // llamado en Update
    void FixedTick(); // llamado en FixedUpdate (física)
    void Exit();
}
```

```
// Controlador de la FSM
public class StateMachine
{
    private IState currentState;

    public void SetState(IState newState)
    {
        currentState?.Exit();
        currentState = newState;
        currentState.Enter();
    }

    public void Tick()      => currentState?.Tick();
    public void FixedTick() => currentState?.FixedTick();
}

// Estado concreto: perseguir al jugador
public class ChaseState : IState
{
    private EnemyAI agent;
    private StateMachine sm;

    public ChaseState(EnemyAI agent, StateMachine sm)
    { this.agent = agent; this.sm = sm; }

    public void Enter()  => agent.Animator.SetTrigger("Chase");
    public void Exit()   => agent.StopMovement();

    public void Tick()
    {
        agent.MoveTowards(agent.Player.position);
        float dist = Vector3.Distance(agent.transform.position,
agent.Player.position);
        if (dist <= agent.AttackRange) sm.SetState(agent.AttackState);
        if (dist > agent.LoseRange)   sm.SetState(agent.SearchState);
    }

    public void FixedTick() { }
}
```

Behavior Trees — Árboles de comportamiento

Un Behavior Tree (BT) es una estructura jerárquica de nodos que modela la toma de decisiones de un agente. A diferencia de la FSM, el BT es modular y reutilizable: los subárboles pueden componerse y compartirse entre distintos tipos de NPC.

Tipos de nodos en un Behavior Tree

Nodo	Tipo	Comportamiento
Sequence	Control	Ejecuta hijos de izquierda a derecha. Si uno falla → falla todo. Éxito solo si todos tienen éxito.
Selector	Control	Ejecuta hijos hasta que uno tiene éxito. Si todos fallan → falla. Como un OR lógico.
Parallel	Control	Ejecuta todos los hijos simultáneamente. Política configurable de éxito/fallo.
Decorator	Control	Modifica el resultado de su único hijo. Ej: Inverter, Repeater, Cooldown.

Condition	Hoja	Evalúa una condición booleana. Retorna Success o Failure inmediatamente.
Action	Hoja	Ejecuta una acción durante uno o más ticks. Retorna Running hasta completar.

```
// Implementación minimal de Behavior Tree
public enum NodeStatus { Success, Failure, Running }

public abstract class BTNode
{
    public abstract NodeStatus Evaluate();
}

// Sequence: falla si cualquier hijo falla
public class Sequence : BTNode
{
    private List<BTNode> children;
    public Sequence(List<BTNode> children) => this.children = children;

    public override NodeStatus Evaluate()
    {
        foreach (var child in children)
        {
            var status = child.Evaluate();
            if (status != NodeStatus.Success) return status;
        }
        return NodeStatus.Success;
    }
}

// Selector: tiene éxito si cualquier hijo tiene éxito
public class Selector : BTNode
{
    private List<BTNode> children;
    public Selector(List<BTNode> children) => this.children = children;

    public override NodeStatus Evaluate()
    {
        foreach (var child in children)
        {
            var status = child.Evaluate();
            if (status != NodeStatus.Failure) return status;
        }
        return NodeStatus.Failure;
    }
}

// Nodo hoja: condición
public class CanSeePlayer : BTNode
{
    private EnemyAI agent;
    public CanSeePlayer(EnemyAI agent) => this.agent = agent;
    public override NodeStatus Evaluate() =>
        agent.HasLineOfSight ? NodeStatus.Success : NodeStatus.Failure;
}

// Nodo hoja: acción
public class ChasePlayer : BTNode
{
    private EnemyAI agent;
    public ChasePlayer(EnemyAI agent) => this.agent = agent;
```

```
    public override NodeStatus Evaluate()
    {
        agent.MoveTowards(agent.PlayerPosition);
        return NodeStatus.Running;
    }
}

// Construcción del árbol de un guardia Stealth
// Selector (raíz)
//   Sequence (combate)
//     CanSeePlayer
//     IsInAttackRange
//     AttackPlayer
//   Sequence (persecución)
//     CanSeePlayer
//     ChasePlayer
//   Sequence (investigación)
//     HasLastKnownPosition
//     MoveToLastKnownPosition
//   PatrolAction
```

FSM vs Behavior Tree — cuándo usar cada uno

Criterio	FSM	Behavior Tree
Complejidad del NPC	Hasta ~6 estados simples	NPCs con muchas condiciones y acciones
Mantenibilidad	Difícil con muchos estados	Alta: subárboles reutilizables y modulares
Debugging	Muy fácil (un estado activo)	Moderado (árbol visual en herramientas como Behavior Designer)
Performance	Muy bajo costo	Bajo-moderado (depende de la profundidad del árbol)
Herramientas Unity	Animator, código manual	Behavior Designer (asset), NodeCanvas, código manual
Recomendado para	Enemigos simples, objetos	Guardias complejos, jefes, aliados con IA avanzada

4.3 Navegación y pathfinding: NavMesh, NavMeshAgent, obstáculos dinámicos y A* conceptual

El pathfinding es la capacidad de un agente para encontrar una ruta válida entre dos puntos evitando obstáculos. Unity implementa pathfinding mediante el sistema NavMesh, que precalcula una malla de navegación sobre la geometría estática de la escena.

NavMesh — malla de navegación

El NavMesh (Navigation Mesh) es una representación simplificada de las superficies transitables de la escena. Se genera automáticamente por Unity (Window → AI → Navigation → Bake) analizando los GameObjects marcados como 'Navigation Static'.

- **Agent Radius:** Radio del agente. Determina el margen de separación con los obstáculos.
- **Agent Height:** Altura mínima de pasajes transitables por el agente.
- **Max Slope:** Pendiente máxima que el agente puede escalar (en grados).

- **Step Height:** Altura máxima de escalones que el agente puede subir sin saltar.
- **NavMesh Areas:** Zonas del NavMesh con distintos costos de travesía (agua = alto costo, carretera = bajo costo). El agente prefiere rutas de menor costo total.

NavMeshAgent — control del agente

```
using UnityEngine.AI;

public class NavAgent : MonoBehaviour
{
    private NavMeshAgent agent;
    [SerializeField] private Transform player;
    [SerializeField] private Transform[] patrolPoints;
    private int currentPatrolIndex;

    void Awake() => agent = GetComponent<NavMeshAgent>();

    // Moverse hacia el jugador
    public void ChasePlayer()
    {
        agent.SetDestination(player.position);
        agent.speed = chaseSpeed;
    }

    // Patrullaje por waypoints
    public void Patrol()
    {
        if (agent.remainingDistance < agent.stoppingDistance + 0.1f)
        {
            currentPatrolIndex = (currentPatrolIndex + 1) % patrolPoints.Length;
            agent.SetDestination(patrolPoints[currentPatrolIndex].position);
        }
    }

    // Verificar si llegó al destino
    public bool ReachedDestination() =>
        !agent.pathPending && agent.remainingDistance <= agent.stoppingDistance;

    // Detener el agente temporalmente
    public void Stop()
    {
        agent.isStopped = true;
        agent.velocity  = Vector3.zero;
    }

    public void Resume() => agent.isStopped = false;

    // Verificar si hay ruta válida antes de moverse
    public bool CanReach(Vector3 target)
    {
        NavMeshPath path = new NavMeshPath();
        return agent.CalculatePath(target, path) &&
               path.status == NavMeshPathStatus.PathComplete;
    }
}
```

NavMesh dinámico con NavMeshObstacle

Los NavMeshObstacles son objetos que modifican el NavMesh en tiempo de ejecución, creando huecos que los agentes evitan. Son ideales para puertas, cajas desplazables, vehículos y cualquier obstáculo que cambie de posición durante el juego.

- **Modo Carve:** El obstáculo 'esculpe' un hueco en el NavMesh cuando está estático. Los agentes replican su ruta alrededor. Si el obstáculo se mueve frecuentemente, desactivar Carve para evitar recálculos costosos.
- **Modo no-Carve:** El obstáculo empuja a los agentes mediante steering local sin modificar el NavMesh. Mejor para objetos en movimiento continuo.

A* — concepto y relación con NavMesh

El algoritmo A* (A-estrella) es el algoritmo de pathfinding más usado en videojuegos. Encuentra la ruta de menor costo entre dos nodos de un grafo combinando el costo real acumulado (g) y una estimación heurística del costo restante (h). Unity usa una variante de A* internamente sobre el NavMesh.

```
// A* conceptual — pseudocódigo comentado
// f(n) = g(n) + h(n)
// g(n) = costo real desde el inicio hasta el nodo n
// h(n) = heurística: distancia estimada de n al destino (ej: distancia
Manhattan)

// 1. Agregar nodo inicial a la lista OPEN con f = 0 + h(inicio)
// 2. Mientras OPEN no esté vacía:
//    a. Extraer nodo con menor f de OPEN → nodo actual
//    b. Si nodo actual == destino → reconstruir ruta y terminar
//    c. Mover nodo actual a CLOSED
//    d. Para cada vecino del nodo actual:
//       - Si en CLOSED → saltar
//       - Calcular g_nuevo = g(actual) + costo(actual, vecino)
//       - Si vecino no en OPEN O g_nuevo < g(vecino):
//           * padre(vecino) = actual
//           * g(vecino) = g_nuevo
//           * f(vecino) = g_nuevo + h(vecino)
//           * Agregar vecino a OPEN

// Heurísticas comunes:
// - Manhattan: |dx| + |dy|  → grillas sin movimiento diagonal
// - Euclidiana: sqrt(dx²+dy²) → espacios continuos (NavMesh)
// - Chebyshev:  max(|dx|,|dy|) → grillas con movimiento diagonal
```

4.4 Sistemas de detección: línea de visión, conos de detección, FOV, estados de alerta y patrullaje

Los sistemas de detección son el puente entre la percepción del agente y su lógica de decisión. Un guardia en un juego de sigilo debe detectar al jugador de forma creíble: no omnisciente (para que el sigilo sea posible) ni ciega (para que suponga un desafío real).

Línea de visión (Line of Sight)

La línea de visión determina si el agente puede ver directamente al objetivo sin obstáculos entre ambos. Se implementa con un Raycast desde el ojo del agente hacia el objetivo, verificando que no haya geometría bloqueante entre ellos.

```
public bool HasLineOfSight(Transform target)
{
    Vector3 dirToTarget = (target.position - eyePosition.position).normalized;
    float   distToTarget = Vector3.Distance(eyePosition.position,
target.position);

    // Raycast ignorando el propio collider del agente
    if (Physics.Raycast(eyePosition.position, dirToTarget, out RaycastHit hit,
                        distToTarget, obstacleMask))
    {
        // El rayo impactó algo antes de llegar al objetivo → bloqueado
        return false;
    }
    return true;
}
```

Cono de detección (FOV — Field of View)

El cono de visión limita la detección al ángulo frontal del agente, simulando la visión periférica humana. Se compone de dos verificaciones: distancia al objetivo y ángulo entre la dirección frontal del agente y la dirección hacia el objetivo.

```
public class DetectionSystem : MonoBehaviour
{
    [Header("FOV Settings")]
    [SerializeField] private float viewRadius   = 12f;  // radio de detección
    [SerializeField] private float viewAngle    = 90f;  // ángulo total del cono
(°)
    [SerializeField] private float hearingRadius= 5f;   // detección auditiva sin
LOS
    [SerializeField] private LayerMask playerMask;
    [SerializeField] private LayerMask obstacleMask;

    public Transform DetectedTarget { get; private set; }

    void Update() => DetectedTarget = FindVisibleTarget();

    Transform FindVisibleTarget()
    {
        // 1. Detectar todos los colliders dentro del radio de visión
        Collider[] targetsInRadius = Physics.OverlapSphere(
            transform.position, viewRadius, playerMask);

        foreach (var t in targetsInRadius)
        {
            Transform target = t.transform;
            Vector3 dirToTarget = (target.position -
transform.position).normalized;

            // 2. Verificar que esté dentro del ángulo del cono
            float angle = Vector3.Angle(transform.forward, dirToTarget);
            if (angle > viewAngle / 2f) continue;

            // 3. Verificar línea de visión sin obstáculos
            float dist = Vector3.Distance(transform.position, target.position);
            if (!Physics.Raycast(transform.position, dirToTarget, dist,
obstacleMask))
                return target;   // objetivo visible
        }

        // Detección auditiva: detectar aunque no esté en el cono
```

```
        Collider[] nearTargets = Physics.OverlapSphere(
            transform.position, hearingRadius, playerMask);
        if (nearTargets.Length > 0) return nearTargets[0].transform;

        return null;
    }

    // Dibujar gizmos del FOV en el editor
    void OnDrawGizmosSelected()
    {
        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(transform.position, viewRadius);

        Vector3 fovLeft  = DirFromAngle(-viewAngle / 2f);
        Vector3 fovRight = DirFromAngle( viewAngle / 2f);
        Gizmos.DrawLine(transform.position, transform.position + fovLeft  *
viewRadius);
        Gizmos.DrawLine(transform.position, transform.position + fovRight *
viewRadius);

        Gizmos.color = Color.red;
        Gizmos.DrawWireSphere(transform.position, hearingRadius);
    }

    Vector3 DirFromAngle(float angleInDegrees)
    {
        float rad = (transform.eulerAngles.y + angleInDegrees) * Mathf.Deg2Rad;
        return new Vector3(Mathf.Sin(rad), 0, Mathf.Cos(rad));
    }
}
```

Estados de alerta

Un sistema de alerta gradual hace que los guardias respondan de forma progresiva al jugador, en lugar de pasar instantáneamente de 'tranquilo' a 'atacando'. Esto crea tensión y da al jugador tiempo para reaccionar.

Estado de alerta	Condición de entrada	Comportamiento del guardia
Patrol	Estado por defecto	Sigue ruta de waypoints, ángulo FOV normal
Suspicious	Escuchó ruido o vio movimiento periférico	Se detiene, mira hacia el estímulo, FOV aumenta
Alert	Detectó al jugador brevemente o oyó disparo	Se dirige a última posición conocida, llama refuerzos
Chase	Tiene línea de visión directa al jugador	Persigue activamente, máxima velocidad
Search	Perdió de vista al jugador	Investiga zona, patrulla aumentada, vuelve a Alert
Combat	Jugador en rango de ataque	Dispara/ataca, busca cobertura, lanza granadas

Patrullaje con waypoints

```
public class PatrolSystem : MonoBehaviour
{
    [SerializeField] private Transform[] waypoints;
    [SerializeField] private float       waitTime    = 2f;
    [SerializeField] private bool        randomOrder = false;

    private NavMeshAgent agent;
    private int   currentIndex;
    private float waitTimer;
    private bool  waiting;

    void Awake() => agent = GetComponent<NavMeshAgent>();

    public void PatrolTick()
    {
        if (waiting)
        {
            waitTimer -= Time.deltaTime;
            if (waitTimer <= 0) { waiting = false; MoveToNext(); }
            return;
        }

        if (agent.remainingDistance <= agent.stoppingDistance)
        {
            waiting   = true;
            waitTimer = waitTime;
            // Girar hacia una dirección aleatoria al llegar (look around)
            StartCoroutine(LookAround());
        }
    }

    void MoveToNext()
    {
        if (randomOrder) currentIndex = Random.Range(0, waypoints.Length);
        else             currentIndex = (currentIndex + 1) %
waypoints.Length;
        agent.SetDestination(waypoints[currentIndex].position);
    }

    IEnumerator LookAround()
    {
        float targetAngle = Random.Range(-60f, 60f);
        Quaternion targetRot = Quaternion.Euler(0, transform.eulerAngles.y +
targetAngle, 0);
        float t = 0;
        while (t < 1f)
        {
            transform.rotation = Quaternion.Slerp(transform.rotation,
targetRot, t);
            t += Time.deltaTime * 1.5f;
            yield return null;
        }
    }
}
```

4.5 IA avanzada: Utility AI, GOAP y steering behaviors para movimiento autónomo

Las técnicas avanzadas de IA permiten crear agentes con comportamientos más ricos, adaptables y menos predecibles que los basados en FSM simples. Se usan cuando la variedad de comportamientos y la capacidad de adaptación son críticas para la experiencia de juego.

Utility AI

La Utility AI asigna una puntuación de utilidad a cada acción disponible y selecciona la de mayor valor en cada frame de decisión. Esto produce comportamientos más variados y naturales que una FSM, ya que la elección depende del contexto cuantificado.

```
public class UtilityAI : MonoBehaviour
{
    private List<AIAction> actions = new();

    void Start()
    {
        // Registrar acciones disponibles con su función de utilidad
        actions.Add(new AIAction("Attack",  () => CalcAttackUtility()));
        actions.Add(new AIAction("Flee",    () => CalcFleeUtility()));
        actions.Add(new AIAction("Heal",    () => CalcHealUtility()));
        actions.Add(new AIAction("Patrol",  () => 0.1f));  // utilidad base
    }

    void Update()
    {
        // Seleccionar la acción de mayor utilidad
        AIAction best = actions.OrderByDescending(a =>
a.ScoreFunc()).First();
        best.Execute();
    }

    float CalcAttackUtility()
    {
        if (!detectionSystem.HasTarget) return 0f;
        float healthFactor  = health / maxHealth;          // más HP → más
agresivo
        float distanceFactor= 1f - (distToPlayer / maxRange); // más cerca →
atacar
        return healthFactor * 0.6f + distanceFactor * 0.4f;
    }

    float CalcFleeUtility()
    {
        float lowHealthFactor = 1f - (health / maxHealth); // menos HP →
huir más
        return lowHealthFactor > 0.7f ? lowHealthFactor : 0f;
    }

    float CalcHealUtility()
    {
        return hasHealItem && health < maxHealth * 0.5f ? 0.9f : 0f;
    }
}

public class AIAction
{
    public string    Name;
    public Func<float> ScoreFunc;
    public AIAction(string name, Func<float> score) { Name = name; ScoreFunc
= score; }
```

```
    public void Execute() { /* lógica de la acción */ }
}
```

GOAP — Goal-Oriented Action Planning

GOAP (Planificación de Acciones Orientada a Objetivos) es una arquitectura donde el agente define un objetivo (goal) y un conjunto de acciones con precondiciones y efectos. El planificador encuentra en tiempo de ejecución la secuencia de acciones que transforma el estado actual del mundo en el estado objetivo.

Fue popularizado por el juego F.E.A.R. (2005), donde los guardias coordinaban ataques de flanqueo, buscaban cobertura y lanzaban granadas de forma autónoma. En Unity se puede implementar manualmente o con librerías como CrashKonijn GOAP.

Concepto GOAP	Descripción	Ejemplo
World State	Conjunto de pares clave-valor que describen el estado del mundo	{'hasWeapon': true, 'playerVisible': false}
Goal	Estado del mundo que el agente quiere alcanzar	{'playerDead': true}
Action	Operación con precondiciones y efectos sobre el world state	Attack: pre={hasAmmo}, effect={playerDead}
Planner	Algoritmo (A*) que encuentra la secuencia de acciones óptima	Reload → MoveTo → Attack

Steering Behaviors

Los steering behaviors son fuerzas vectoriales que guían el movimiento de un agente de forma fluida y autónoma. Craig Reynolds los formalizó en 1999. Son ideales para movimiento de multitudes, bandadas (boids), convoys y comportamientos de evasión suave.

Behavior	Descripción	Fórmula simplificada
Seek	Moverse hacia un objetivo	desired = normalize(target - pos) * maxSpeed; steer = desired - velocity
Flee	Alejarse de un objetivo	desired = normalize(pos - target) * maxSpeed; steer = desired - velocity
Arrival	Seek con desaceleración al llegar	speed = maxSpeed * (dist / slowRadius) cuando dist < slowRadius
Wander	Movimiento errático natural	target = posición en círculo proyectado con offset angular aleatorio
Separation	Evitar colisión con vecinos cercanos	steer = suma de vectores de alejamiento de cada vecino
Cohesion	Mantenerse cerca del centro del grupo	steer = seek hacia el centroide del grupo
Alignment	Alinear velocidad con vecinos	steer = velocidad promedio del grupo - velocidad propia

```
public class SteeringAgent : MonoBehaviour
{
    [SerializeField] private float maxSpeed   = 5f;
    [SerializeField] private float maxForce   = 10f;
```

```
    [SerializeField] private float slowRadius = 3f;

    private Rigidbody rb;
    private Vector3   velocity;

    void Awake() => rb = GetComponent<Rigidbody>();

    public Vector3 Seek(Vector3 target)
    {
        Vector3 desired = (target - transform.position).normalized * maxSpeed;
        return Vector3.ClampMagnitude(desired - velocity, maxForce);
    }

    public Vector3 Arrival(Vector3 target)
    {
        Vector3 toTarget = target - transform.position;
        float   dist     = toTarget.magnitude;
        float   speed    = dist < slowRadius ? maxSpeed * (dist / slowRadius) :
maxSpeed;
        Vector3 desired  = toTarget.normalized * speed;
        return Vector3.ClampMagnitude(desired - velocity, maxForce);
    }

    public Vector3 Flee(Vector3 threat)
    {
        Vector3 desired = (transform.position - threat).normalized * maxSpeed;
        return Vector3.ClampMagnitude(desired - velocity, maxForce);
    }

    public void ApplyForce(Vector3 force)
    {
        velocity = Vector3.ClampMagnitude(velocity + force, maxSpeed);
        rb.linearVelocity = velocity;
        if (velocity.sqrMagnitude > 0.01f)
            transform.forward = Vector3.Slerp(transform.forward,
velocity.normalized, Time.fixedDeltaTime * 8f);
    }
}
```

4.6 Prototipo 3: Juego Stealth/Acción con NPCs inteligentes — patrullaje, detección, persecución y combate

El Prototipo 3 es el integrador de la Unidad 4. Combina todos los sistemas de IA vistos (FSM/BT, NavMesh, FOV, sistema de alerta, steering behaviors) con los sistemas de U2 (inventario, UI, escenas) y U3 (animaciones, audio, VFX) en un juego de sigilo o acción con NPCs creíbles.

Especificación del prototipo

Sistema	Requisito mínimo	Tecnología
Movimiento del jugador	Caminar (agachado reduce ruido), correr, interactuar	CharacterController o Rigidbody + Input System
Guardia — Patrullaje	Ruta de waypoints con parada y mirada alrededor	NavMeshAgent + PatrolSystem
Guardia — Detección	Cono FOV + radio auditivo + línea de visión	DetectionSystem + Raycast
Guardia — Alerta	5 estados: Patrol→Suspicious→Alert→Chase→Search	FSM con IState o Behavior Tree

Guardia — Combate	Buscar cobertura, disparar, retroceder si bajo HP	NavMesh + Utility AI o BT
Comunicación entre NPCs	Guardia alerta a compañeros en radio de comunicación	Observer con GameEvent SO
Sistema de ruido	Acciones del jugador generan niveles de ruido detectables	Variable float de noise level
UI de alerta	Indicador visual del nivel de alerta del guardia más cercano	Canvas + Image fill radial
Reinicio y checkpoints	Reiniciar desde último checkpoint al ser detectado	SceneManager + SaveSystem

Sistema de ruido del jugador

```
public class NoiseSystem : MonoBehaviour
{
    public static NoiseSystem Instance { get; private set; }
    public float CurrentNoise { get; private set; }

    [SerializeField] private float decayRate = 2f;  // ruido baja X unidades/seg

    void Awake() { Instance = this; }

    void Update()
    {
        CurrentNoise = Mathf.Max(0, CurrentNoise - decayRate * Time.deltaTime);
    }

    // Emitir ruido desde una posición (pasos, disparos, explosiones)
    public void EmitNoise(Vector3 origin, float amount)
    {
        CurrentNoise = Mathf.Min(100f, CurrentNoise + amount);
        // Notificar a guardias en radio de escucha
        Collider[] guards = Physics.OverlapSphere(origin, amount * 0.5f,
guardLayer);
        foreach (var g in guards)
            g.GetComponent<GuardAI>()?.OnHearNoise(origin, amount);
    }
}

// En PlayerController — emitir ruido según acción
void HandleMovement()
{
    if (isSprinting)
    {
        NoiseSystem.Instance.EmitNoise(transform.position, 15f);
        if (isGrounded) AudioManager.Instance.PlaySFX("footstep_run");
    }
    else if (isMoving && !isCrouching)
    {
        NoiseSystem.Instance.EmitNoise(transform.position, 5f);
    }
    // Agachado y caminando: ruido mínimo, sin emisión
}
```

Arquitectura del guardia — integración de todos los sistemas

```
public class GuardAI : MonoBehaviour
{
    // Subsistemas
    private StateMachine    stateMachine;
    private NavMeshAgent    navAgent;
    private DetectionSystem detection;
    private PatrolSystem    patrol;
    private Animator        anim;

    // Estados
    public IState PatrolState    { get; private set; }
    public IState SuspiciousState{ get; private set; }
    public IState AlertState     { get; private set; }
    public IState ChaseState     { get; private set; }
    public IState SearchState    { get; private set; }
    public IState CombatState    { get; private set; }

    // Datos compartidos entre estados
    public Transform PlayerTransform   { get; private set; }
    public Vector3   LastKnownPosition { get; set; }
    public float     AlertLevel        { get; set; }  // 0..1

    void Awake()
    {
        navAgent  = GetComponent<NavMeshAgent>();
        detection = GetComponent<DetectionSystem>();
        patrol    = GetComponent<PatrolSystem>();
        anim      = GetComponent<Animator>();

        stateMachine   = new StateMachine();
        PatrolState    = new PatrolState(this, stateMachine);
        SuspiciousState= new SuspiciousState(this, stateMachine);
        AlertState     = new AlertState(this, stateMachine);
        ChaseState     = new ChaseState(this, stateMachine);
        SearchState    = new SearchState(this, stateMachine);
        CombatState    = new CombatState(this, stateMachine);

        stateMachine.SetState(PatrolState);
    }

    void Update()      => stateMachine.Tick();
    void FixedUpdate() => stateMachine.FixedTick();

    // Llamado por NoiseSystem cuando hay ruido audible
    public void OnHearNoise(Vector3 origin, float amount)
    {
        if (stateMachine.CurrentState == PatrolState)
        {
            LastKnownPosition = origin;
            stateMachine.SetState(SuspiciousState);
        }
    }

    // Llamado por DetectionSystem al ver al jugador
    public void OnPlayerDetected(Transform player)
    {
        PlayerTransform    = player;
        LastKnownPosition  = player.position;
        stateMachine.SetState(ChaseState);
        // Alertar a guardias cercanos via Observer
        onPlayerDetectedEvent.Raise();
    }
```

```
}
```

Estructura de carpetas del Prototipo 3

```
Assets/_Project/Scripts/
├── AI/
│   ├── Core/
│   │   ├── StateMachine.cs         // Controlador FSM genérico
│   │   ├── IState.cs               // Interfaz base de estado
│   │   └── BTNode.cs               // Nodo base de Behavior Tree
│   ├── States/
│   │   ├── PatrolState.cs          // Patrullaje por waypoints
│   │   ├── SuspiciousState.cs      // Investigar estímulo
│   │   ├── AlertState.cs           // Ir a última posición conocida
│   │   ├── ChaseState.cs           // Perseguir al jugador
│   │   ├── SearchState.cs          // Buscar al jugador tras perderle
│   │   └── CombatState.cs          // Atacar con cover y utility
│   ├── GuardAI.cs                  // Coordinador: referencia todos los
subsistemas
│   ├── DetectionSystem.cs          // FOV + LOS + hearing radius
│   ├── PatrolSystem.cs             // Waypoints, wait, LookAround
│   └── SteeringAgent.cs            // Seek, Flee, Arrival, Wander
├── Player/
│   ├── StealthController.cs        // Movimiento, crouch, ruido
│   └── NoiseSystem.cs              // Singleton: nivel de ruido global
├── Environment/
│   ├── CoverPoint.cs               // Marcador de posición de cobertura
│   └── Checkpoint.cs               // Guarda posición de respawn
└── UI/
    ├── AlertIndicator.cs           // HUD: nivel de alerta del guardia más
cercano
    └── StealthHUD.cs               // Mapa, checkpoints, indicadores de ruido
```

CONSEJO — DISEÑO DE NIVELES PARA SIGILO

Un buen nivel de sigilo tiene múltiples rutas alternativas para cada objetivo: al menos una ruta de sigilo total (sin enemigos), una ruta de riesgo medio (un enemigo evitable) y una ruta de alto riesgo con mayor recompensa. Los conos de visión de los guardias deben ser visibles para el jugador (indicador visual en HUD o representación en el mundo). Sin información clara sobre qué detecta el guardia, el sigilo se convierte en adivinanza, no en habilidad.